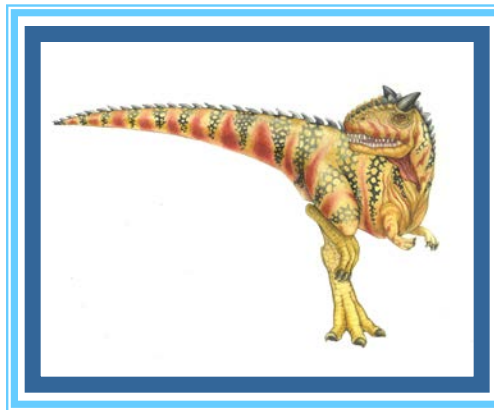


Chapter 8: Main Memory





Chapter 8: Memory Management

- Background
- Swapping
- Contiguous Memory Allocation
- Segmentation
- Paging
- Structure of the Page Table
- Example: The Intel 32 and 64-bit Architectures
- Example: ARM Architecture





Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques, including paging and segmentation
- To provide a detailed description of the Intel Pentium, which supports both pure segmentation and segmentation with paging





Background

- Program must be brought (from disk) into memory and placed within a process for it to be run
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of addresses + read requests, or address + data and write requests
- Register access in one CPU clock (or less)
- Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
- Protection of memory required to ensure correct operation





Base and Limit Registers

- A pair of **base and limit registers** define the logical address space
- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user

Base Register

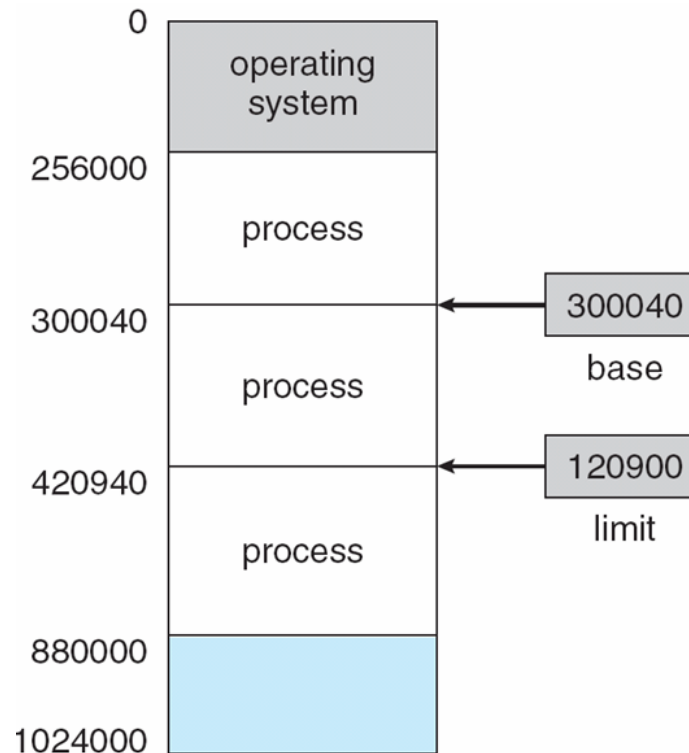
Holds the starting address of the memory area allocated to a process.

Ensures the process cannot access memory below this address.

Limit Register

Holds the size (length) of the memory area allocated to the process.

Ensures the process cannot access memory beyond this range.

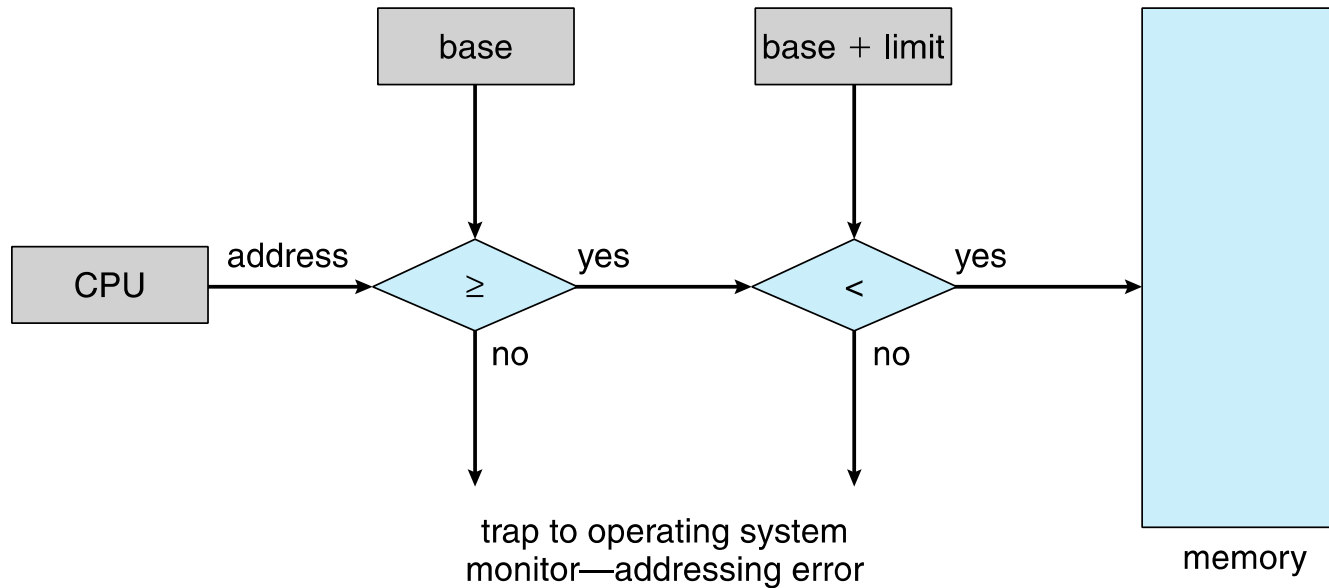


Base and limit registers are used in operating systems to protect and manage a process's memory in main memory.





Hardware Address Protection





Address Binding

- Address binding is the process by which logical (virtual) addresses, generated by the CPU, are translated to physical addresses, which are the actual locations in the computer's RAM
- Further, addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic
 - Compiled code addresses **bind** to relocatable addresses
 - ▶ i.e. "14 bytes from beginning of this module"
 - Linker or loader will bind relocatable addresses to absolute addresses
 - ▶ i.e. 74014
 - Each binding maps one address space to another





Binding of Instructions and Data to Memory

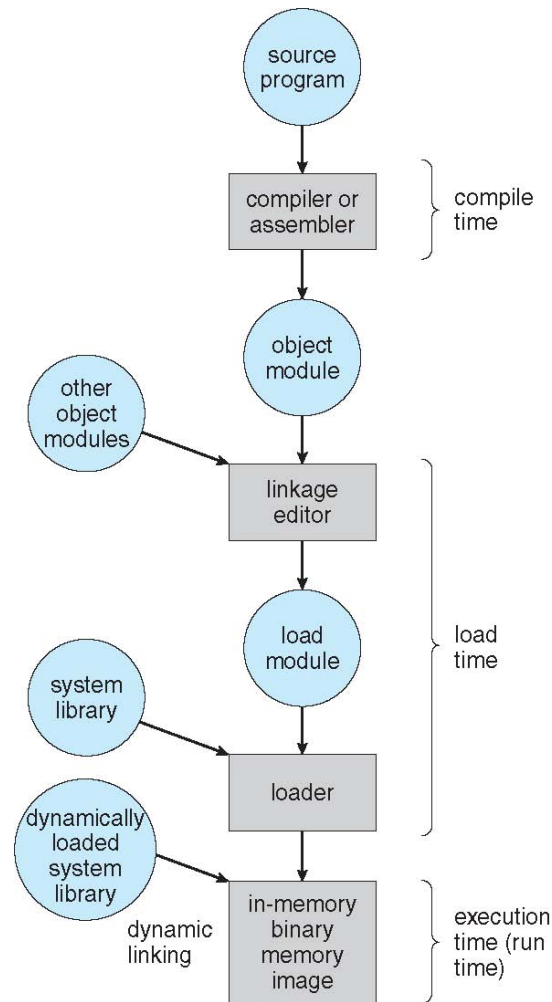
■ Types of address binding in operating systems.

- **Compile time:** Addresses are fixed by the compiler before execution. The program must know where in memory it will be loaded. It is fast during execution but inflexible. Example: MOV AX, 5000 → 5000 is fixed.
- **Load time:** Binding occurs when the program is loaded into memory. The OS adjusts addresses based on the location where the program is loaded. It is flexible, as the program can be loaded anywhere in memory.
- **Execution time:** Binding occurs while the program is running. It requires hardware support, usually a base register, and allows programs to move in memory during execution (relocation).





Multistep Processing of a User Program





Logical vs. Physical Address Space

- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
- **Logical address space** is the set of all logical addresses generated by a program.
 - Also called **virtual address space**.
 - Generated by the **CPU** during program execution.
 - The program **sees memory as a continuous block**, starting from address 0.
 - Independent of where the program is actually loaded in main memory.
 - Example: In a program, instructions like `MOV AX, 500` refer to **logical addresses**.
- **Physical address space** is the set of all physical addresses generated by a program
 - Actual addresses in **main memory (RAM)**.
 - Determined by the **memory hardware** and **OS memory management**.
 - May be different from logical addresses because of **relocation, paging, or segmentation**.
 - Example: The logical address 500 may be mapped to **physical address 1500** in RAM.





Memory-Management Unit (MMU)

- The MMU is a hardware device between the CPU and main memory that is responsible for translating logical addresses into physical addresses. A relocation register is a special register used by the Memory Management Unit (MMU) to map a program's logical addresses into physical addresses at run time.

◆ Main Functions of MMU

1. Address Translation

- Converts **logical (virtual) addresses** generated by the CPU into **physical addresses** in RAM.
- Example: CPU generates address `0x2000` → MMU maps it to physical location `0xAB2000` in RAM.

2. Protection

- Ensures one process cannot access another process's memory.
- Uses **base and limit registers** or **page tables**.

3. Relocation

- Allows processes to be moved around in memory without changing their logical addresses.

4. Efficient Memory Use

- Supports **paging** and **segmentation** to manage memory effectively.

◆ How It Works (Simple Flow)

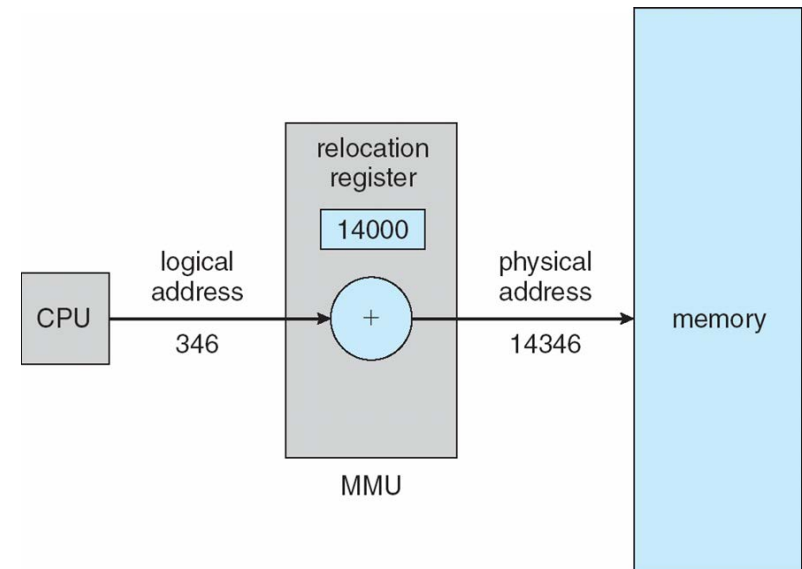
1. CPU generates a **logical address**.
2. Address goes to the **MMU**.
3. MMU translates it into a **physical address**.
4. Physical address is sent to **RAM** to fetch/store data.





Dynamic relocation using a relocation register

- A routine (function or procedure) is only loaded into memory when it is actually called.
- This saves memory because unused routines are never loaded.
- All routines kept on disk in relocatable load format
- This is especially useful when a program has large code for rare cases that may not always run.
- No special support from the operating system is required
 - Implemented through program design
 - OS can help by providing libraries to implement dynamic loading



Dynamic loading means a program loads routines into memory only when needed, saving memory and allowing flexible program execution.





Dynamic Linking

When a program is compiled, it often uses functions stored in libraries (e.g., printf, sqrt). Linking means connecting the program with these library functions so the program can run.

- **Static linking** – The program code and library code are combined into one single file (the executable) before running.
- **Dynamic linking** – Dynamic linking is a process in which the linking of library routines to a program is performed at execution time rather than at compile time. The program contains only references (stubs) to the shared libraries, and the actual code is loaded into memory and linked when the program is executed.
- A small helper piece of code, called a stub, is kept in the program. The stub's job is to find the library function in memory. Once found, the stub replaces itself with the function's address and then runs it.
- The operating system checks if the needed library function is already in the program's memory space.
 - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**
- Consider applicability to patching system libraries
 - Versioning may be needed





Swapping

- Swapping means moving a process (or part of it) from main memory to secondary storage (usually disk) and later bringing it back to main memory for execution. A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
 - This allows the total size of all processes to be larger than the physical RAM.
- **Backing store** – The backing store is a fast disk area big enough to keep copies of all processes. It must allow direct access to any process when needed.
- **Roll out, roll in** – Roll out, roll in is a special type of swapping used in priority scheduling:
 - A low-priority process is moved out (rolled out).
 - A high-priority process is brought in (rolled in).
- The main time cost in swapping comes from transferring data between RAM and disk. The more memory swapped, the longer it takes.
- The system keeps a ready queue of processes that are ready to run but currently stored on disk.





Swapping (Cont.)

■ How It Works

Process in Memory: A process is loaded into RAM for execution.

Memory Shortage: If RAM becomes full and another process needs to be executed, the OS selects a process and moves it temporarily to disk (called swap space).

Swapping Out: The chosen process is written to the swap space.

Swapping In: When that process is needed again, it is brought back from disk into RAM.

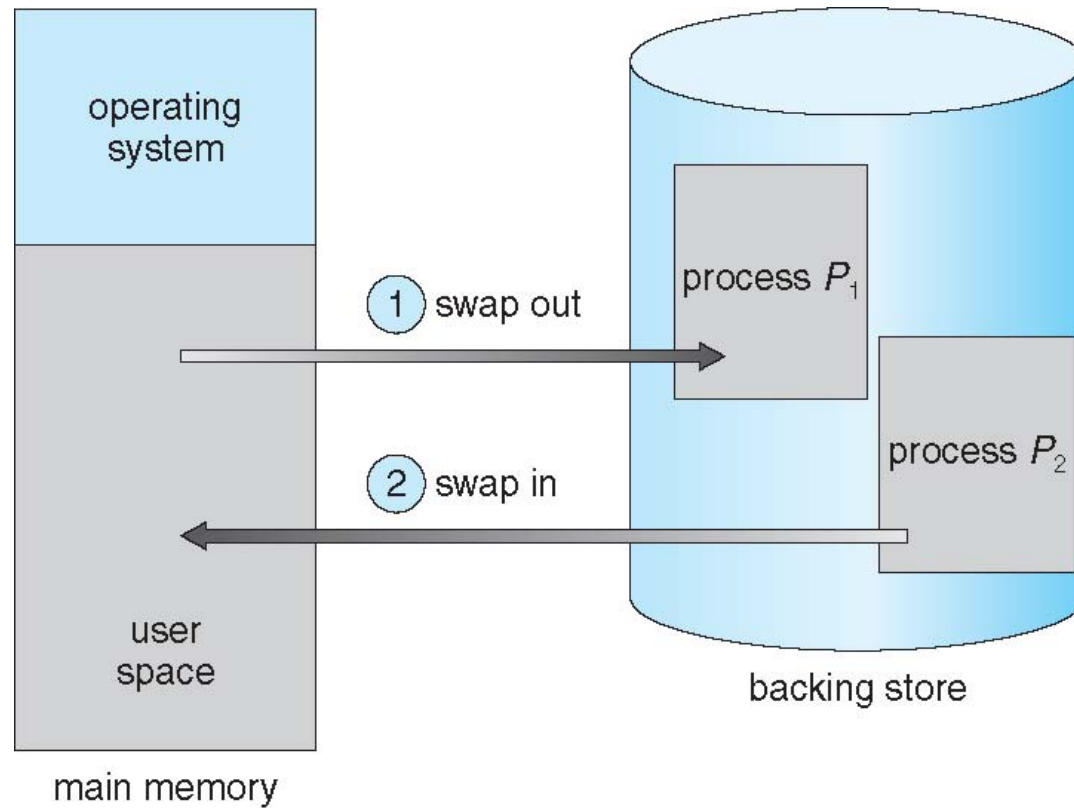
■ Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)

- Swapping normally disabled
- Started if more than threshold amount of memory allocated
- Disabled again once memory demand reduced below threshold





Schematic View of Swapping





Context Switch Time including Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
 - Swap out time of 2000 ms
 - Plus swap in of same sized process
 - Total context switch swapping component time of 4000ms (4 seconds)
- Can reduce if reduce size of memory swapped – by knowing how much memory really being used
 - System calls to inform OS of memory use via `request_memory()` and `release_memory()`





Context Switch Time and Swapping (Cont.)

- Other constraints as well on swapping
 - Pending I/O – can't swap out as I/O would occur to wrong process
 - Or always transfer I/O to kernel space, then to I/O device
 - ▶ Known as **double buffering**, adds overhead
- Standard swapping not used in modern operating systems
 - But modified version common
 - ▶ Swap only when free memory extremely low





Swapping on Mobile Systems

- Not typically supported
 - Flash memory based
 - ▶ Small amount of space
 - ▶ Limited number of write cycles
 - ▶ Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
 - iOS **asks** apps to voluntarily relinquish allocated memory
 - ▶ Read-only data thrown out and reloaded from flash if needed
 - ▶ Failure to free can result in termination
 - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
 - Both OSes support paging as discussed below





Contiguous Allocation

Contiguous allocation is a memory management method where each process is allocated a single continuous block of memory. The process occupies memory addresses that are next to each other.

How It Works

When a process arrives, the OS searches for a single large enough free block to fit it.

Once a suitable block is found, the process is loaded into that contiguous space.

The process executes in that block until it finishes or is swapped out.

Advantages

Simple and fast memory access because addresses are contiguous.

Easy to calculate physical addresses using a base (relocation) register

.





Contiguous Allocation (Cont.)

Disadvantages

External fragmentation occurs as free memory gets split into holes.

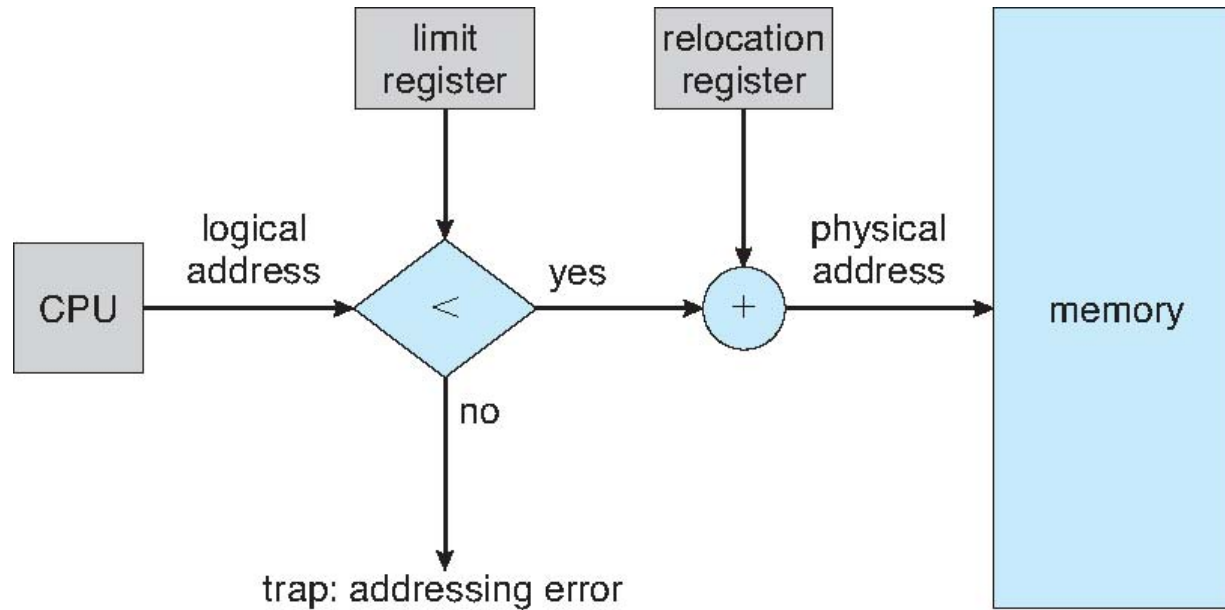
Memory allocation can be wasteful if the process is smaller than the block.

Hard to increase process size during execution (limited flexibility).





Hardware Support for Relocation and Limit Registers

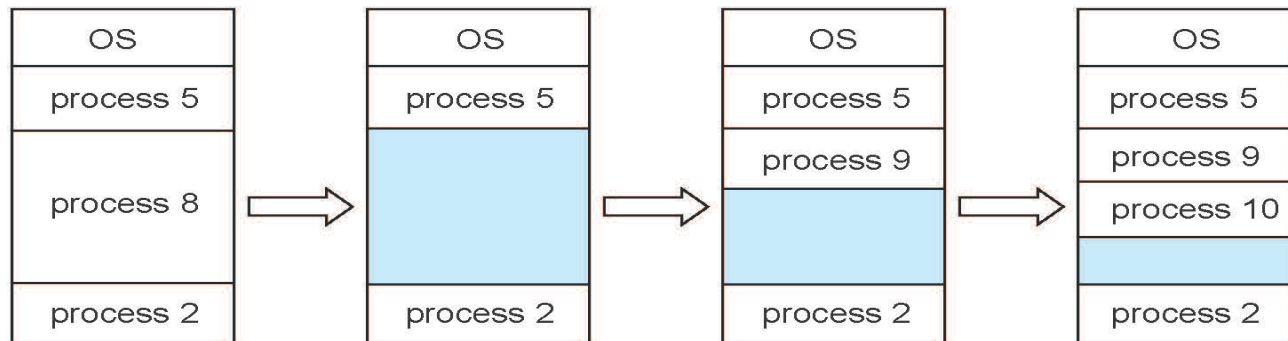




Multiple-partition allocation

■ Multiple-partition allocation is a memory management scheme where the main memory is divided into several partitions, and each partition can hold one process.

- The number of partitions limits the degree of multiprogramming (how many processes can run at the same time).
- Variable-size partitions are used for efficiency → each partition is sized according to the process's memory needs.
- A hole is a block of free memory. There can be many holes of different sizes scattered in memory.
- When a process arrives, it is allocated memory from a hole that is big enough to hold it.
- When a process finishes, its partition is freed, and adjacent free partitions (holes) are combined to make a bigger free block.
- The operating system keeps track of:
 1. Allocated partitions (which partitions are in use)
 2. Free partitions (holes)





Dynamic Storage-Allocation Problem

Memory allocation strategies

- **First-fit:** Allocate the **first** hole that is big enough. Fast because it stops searching as soon as it finds a fit.
- **Best-fit:** Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size. Can reduce wasted memory but may create many tiny unusable holes.
 - Produces the smallest leftover hole
- **Worst-fit:** Allocate the **largest** hole; must also search entire list
 - Produces the largest leftover hole

First-fit and best-fit better than worst-fit in terms of speed and storage utilization





Fragmentation

Fragmentation is a situation in memory management where free memory is broken into small pieces and cannot be used efficiently, even if the total free memory is enough for a process

- **External Fragmentation** – There is enough total free memory to meet a process's request, but the free memory is scattered in small pieces, so the process cannot be loaded.
- **Internal Fragmentation** – The memory given to a process is slightly bigger than it actually needs, and the unused portion inside the allocated block is wasted.
- **First fit analysis** - If N blocks are allocated using first-fit, approximately $0.5 \times N$ blocks may be lost to fragmentation.
 - 1/3 may be unusable -> **50-percent rule**





Fragmentation (Cont.)

- Reduce external fragmentation by **compaction**
 - Compaction means shuffling memory contents so that all free memory is combined into one large block.
 - Compaction is possible *only* if relocation is dynamic, and is done at execution time
 - I/O problem
 - ▶ A process involved in I/O cannot be moved easily in memory
 - ▶ Solution: Do I/O operations only using OS buffers, not directly to the process's memory.





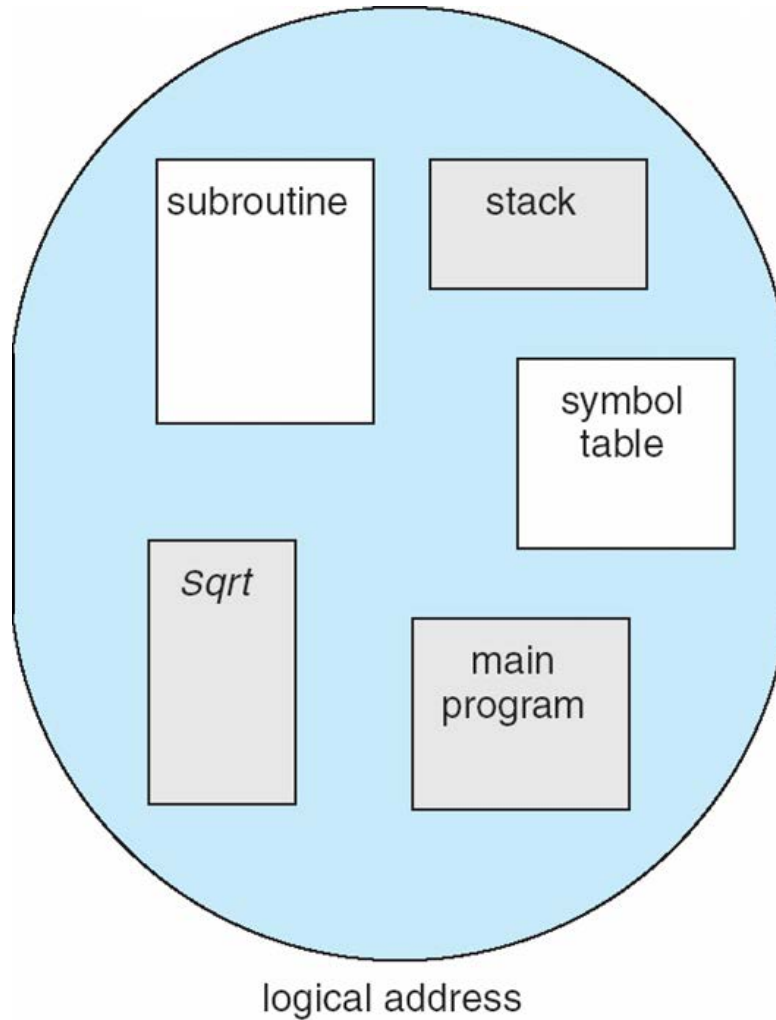
Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments
 - A segment is a logical unit such as:
 - main program
 - procedure
 - function
 - method
 - object
 - local variables, global variables
 - common block
 - stack
 - symbol table
 - arrays



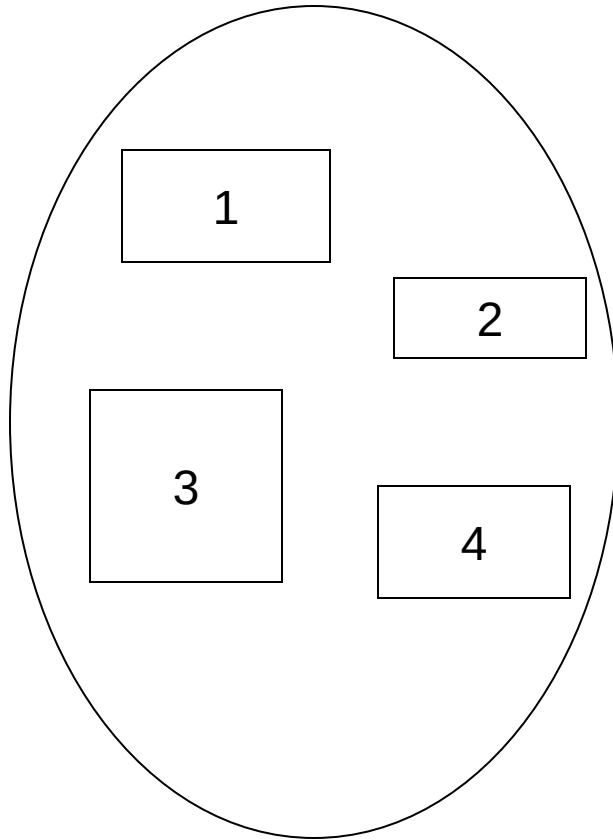


User's View of a Program

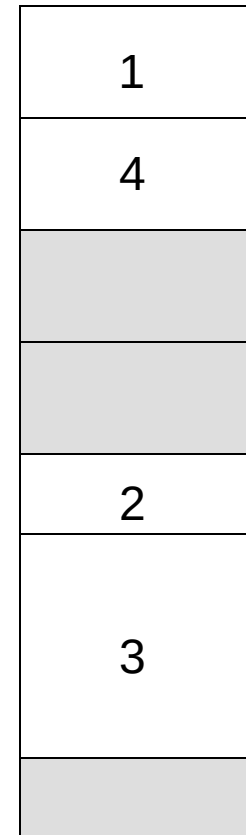




Logical View of Segmentation



user space



physical memory space





Segmentation Architecture

- Logical address consists of a two tuple:
 <segment-number, offset>,
- **Segment table** – maps two-dimensional physical addresses; each table entry has:
 - **base** – contains the starting physical address where the segments reside in memory
 - **limit** – specifies the length of the segment
- **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;
 segment number **s** is legal if **s** < **STLR**





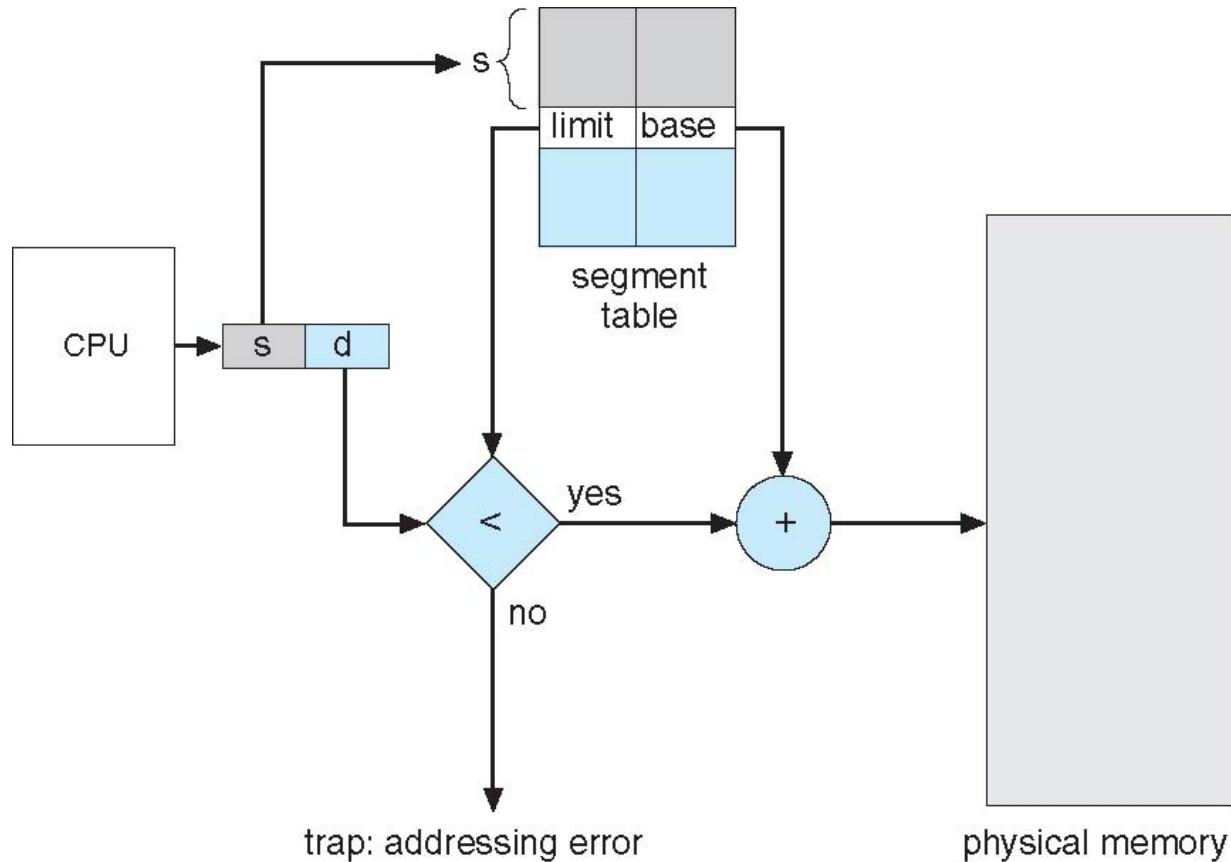
Segmentation Architecture (Cont.)

- Protection
 - With each entry in segment table associate:
 - ▶ validation bit = 0 $\Rightarrow$ illegal segment
 - ▶ read/write/execute privileges
- Protection bits associated with segments; code sharing occurs at segment level
- Since segments vary in length, memory allocation is a dynamic storage-allocation problem
- A segmentation example is shown in the following diagram





Segmentation Hardware





Paging

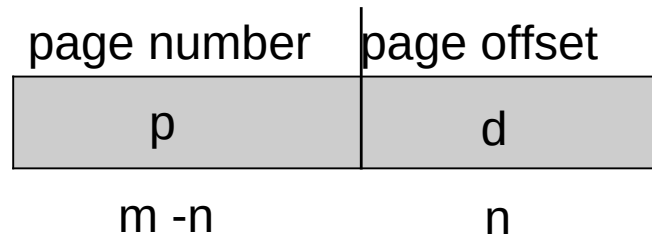
- Paging is a memory management scheme that divides both physical memory (RAM) and logical memory (process address space) into fixed-size blocks so that processes can be loaded non-contiguously, eliminating the problem of external fragmentation.
 - Avoids external fragmentation
 - Avoids problem of varying sized memory chunks
- A fixed-size block of logical memory called **frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes
- A fixed-size block of physical memory of the same size called **pages**
- Keep track of all free frames
- To run a program of size N pages, need to find N free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation





Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number (p)** – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset (d)** – This part gives the exact location inside the page. It is added to the page's base address from the page table to get the physical memory address.

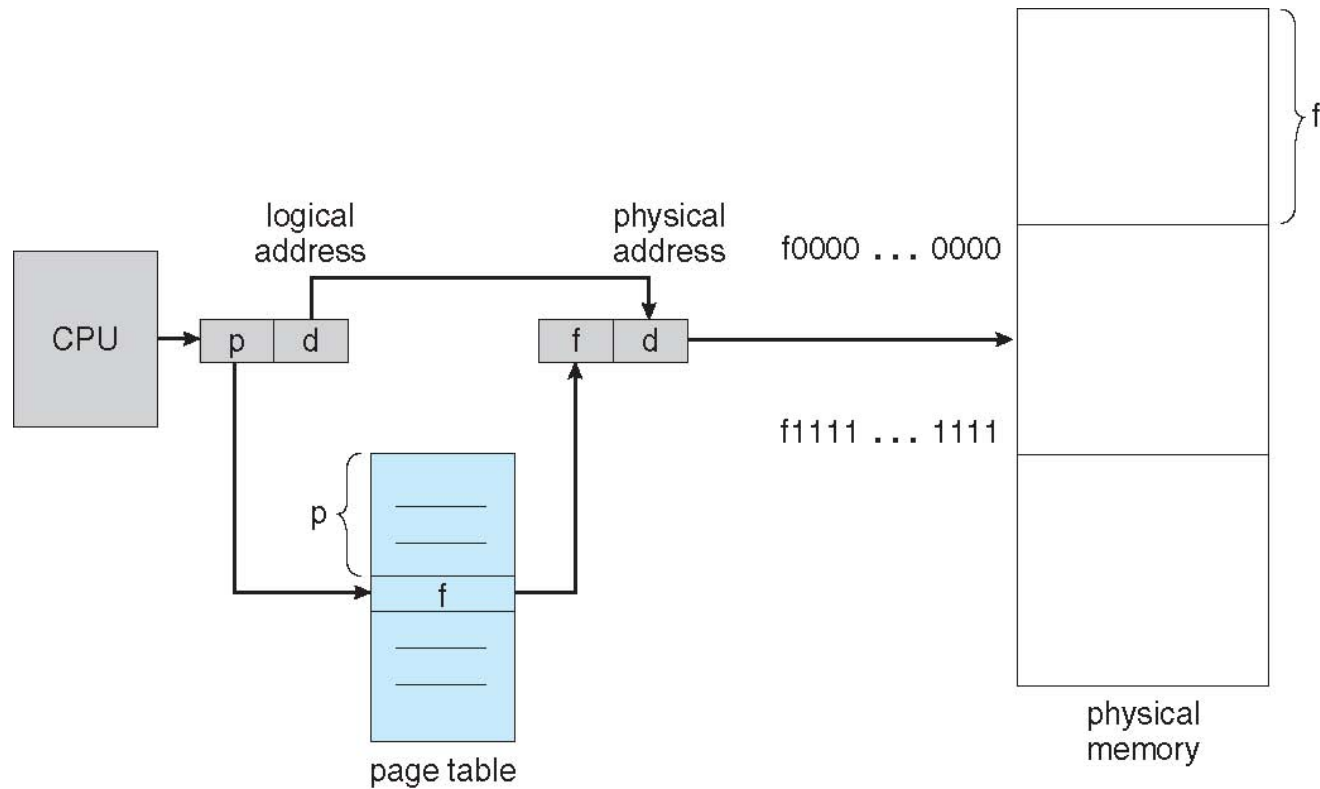


- For given logical address space 2^m and page size 2^n



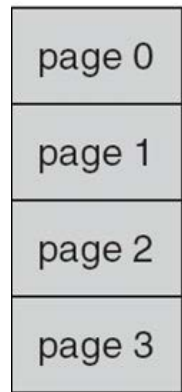


Paging Hardware





Paging Model of Logical and Physical Memory

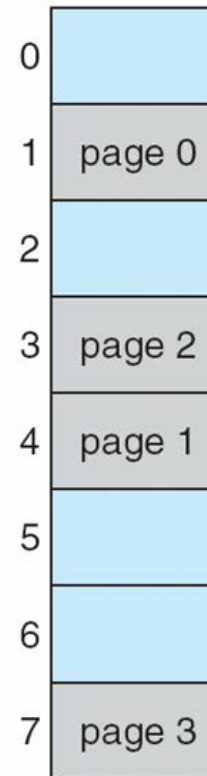


logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

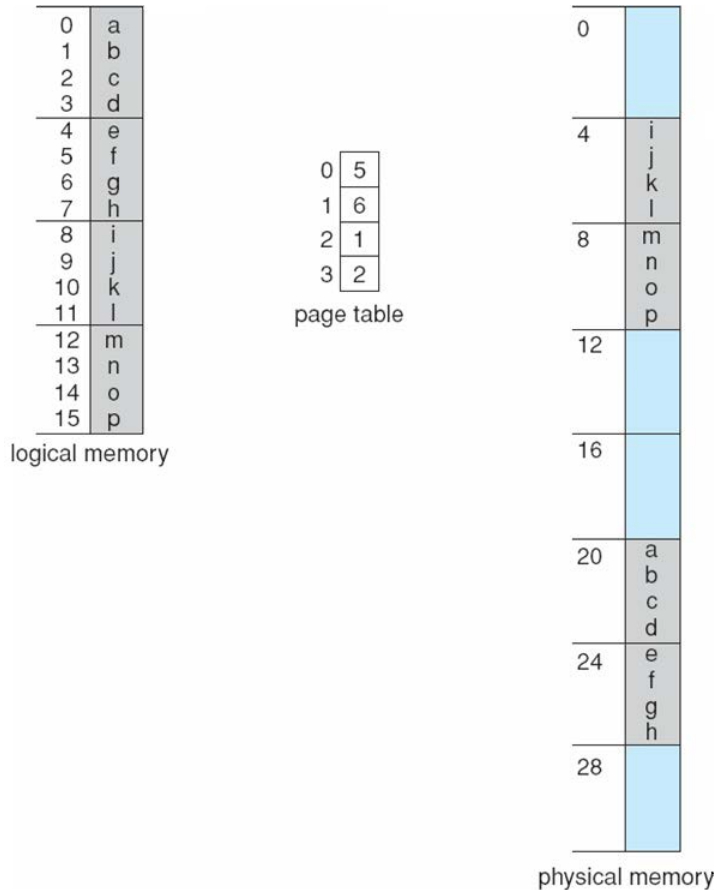


physical
memory





Paging Example



$n=2$ and $m=4$ 32-byte memory and 4-byte pages





Paging (Cont.)

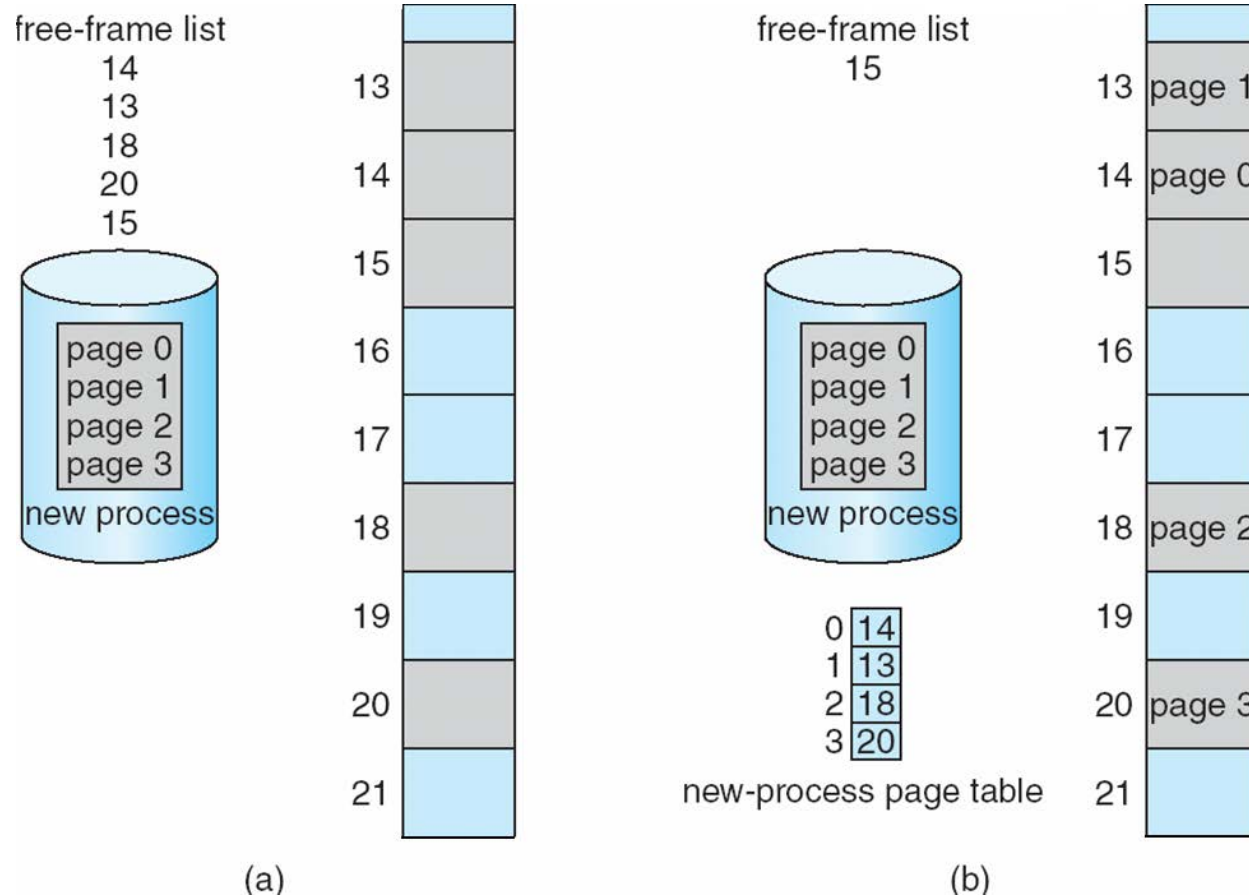
◆ Calculating Internal Fragmentation in Paging

- **Example:**
 - Page size = 2,048 bytes
 - Process size = 72,766 bytes
- **Step 1:** Divide process into pages:
 - $72,766 \div 2,048 = 35 \text{ full pages} + 1,086 \text{ bytes remaining}$
- **Step 2:** Internal fragmentation:
 - The last page is **not fully used** → $2,048 - 1,086 = 962 \text{ bytes wasted}$
- **Worst-case fragmentation:**
 - Almost **1 frame wasted** (if only 1 byte is used in the last frame)
- **Average fragmentation:**
 - About **half of a frame size** on average → internal fragmentation $\approx \text{frame size} \div 2$
- **Trade-off:**
 - **Smaller page/frame sizes** → less internal fragmentation
 - **But** smaller pages → **larger page tables**, which also use memory
- **Modern systems:**
 - Page sizes have grown over time
 - Example: Solaris supports **8 KB** and **4 MB** pages
 - The process's **view of memory** is now very different from **physical memory**
 - OS ensures a process can **only access its own memory**





Free Frames



Before allocation

After allocation





Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register (PTBR)** points to the page table
- **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**





Implementation of Page Table (Cont.)

- Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
 - Otherwise need to flush at every context switch
- TLBs typically small (64 to 1,024 entries)
- On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
 - Some entries can be **wired down** for permanent fast access





Associative Memory

■ Associative memory – parallel search

Page #	Frame #

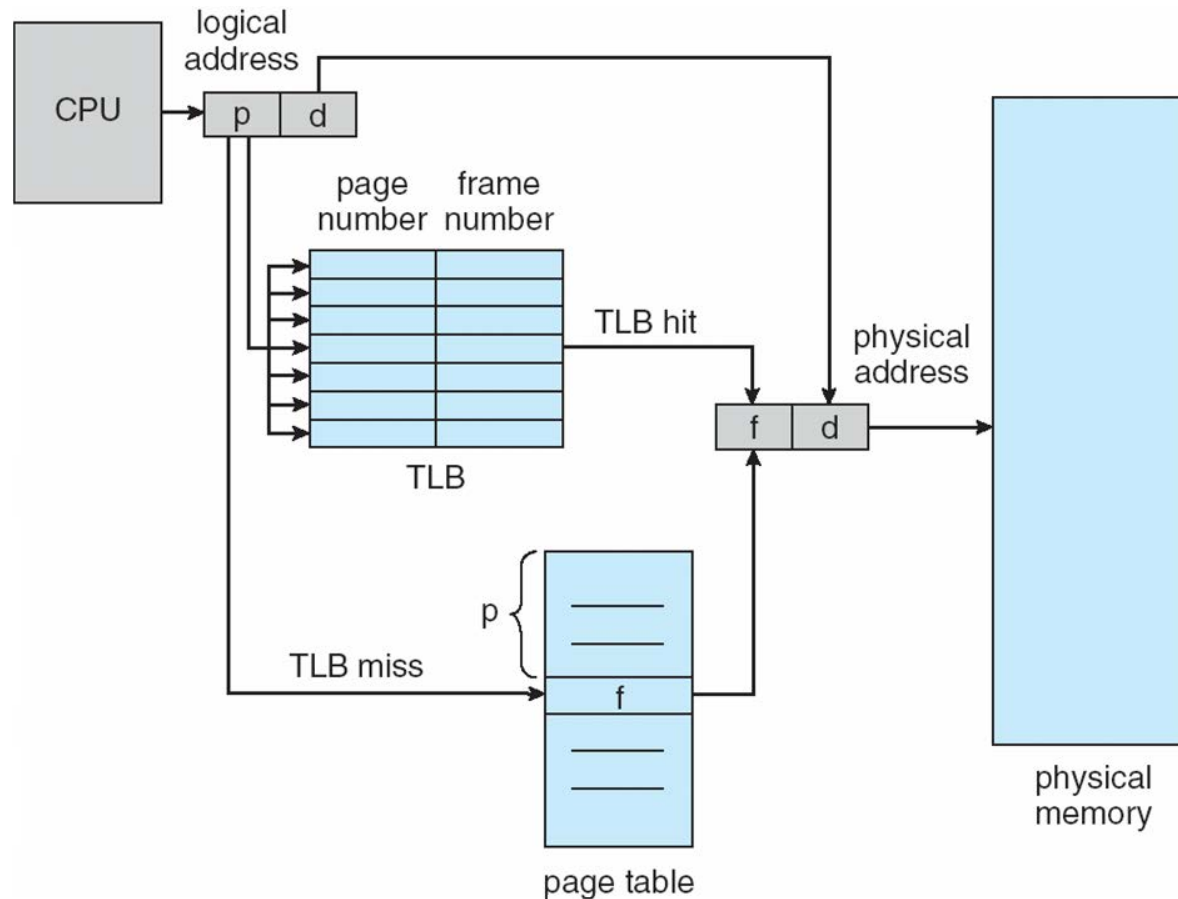
■ Address translation (p, d)

- If p is in associative register, get frame # out
- Otherwise get frame # from page table in memory





Paging Hardware With TLB





Effective Access Time

Effective Access Time is the average time it takes to access memory, taking into account both fast memory (like cache) and slower memory (like main memory or disk).

If you have **cache memory**, the EAT is calculated as:

$$\text{EAT} = (\text{hit ratio} \times \text{cache access time}) + (\text{miss ratio} \times \text{memory access time})$$

Where:

- **Hit ratio (h)** = fraction of memory accesses found in cache
- **Miss ratio** = $1 - h$
- **Cache access time (T_c)** = time to access cache
- **Memory access time (T_m)** = time to access main memory





Effective Access Time

If the page number is found in the TLB (**hit**):

- Time = $\varepsilon + m$

If not found in the TLB (**miss**):

- Time = $\varepsilon + (m + m) = \varepsilon + 2m$ (because first we access page table, then memory again).

So the **Effective Access Time (EAT)**:

$$EAT = \alpha(\varepsilon + m) + (1 - \alpha)(\varepsilon + 2m)$$

- ◆ **Example 1** ($\alpha = 80\%$, $\varepsilon = 20ns$, $m = 100ns$)

$$\begin{aligned} EAT &= 0.80(20 + 100) + 0.20(20 + 200) \\ &= 0.80(120) + 0.20(220) \\ &= 96 + 44 = 140ns \end{aligned}$$

✓ Correct EAT = **140 ns** (not 120 ns as in your note).

- ◆ **Example 2** ($\alpha = 99\%$, $\varepsilon = 20ns$, $m = 100ns$)

$$\begin{aligned} EAT &= 0.99(20 + 100) + 0.01(20 + 200) \\ &= 0.99(120) + 0.01(220) \\ &= 118.8 + 2.2 = 121ns \end{aligned}$$





Memory Protection

- Memory protection is a mechanism used by the operating system to control which parts of memory a process can access, preventing one process from accidentally or maliciously accessing another process's memory.
- **Valid-invalid** The valid–invalid bit is a hardware mechanism used by the operating system to protect memory and check the validity of each memory access.
 - Each entry in a page table has a valid–invalid bit:
 - Valid (V) bit = 1 → The page is in the process's address space and access is allowed.
 - Invalid (I) bit = 0 → The page is not in the process's address space; access is illegal.
 - Or use **page-table length register (PTLR)**
- Any violations result in a trap to the kernel





Valid (v) or Invalid (i) Bit In A Page Table

00000	page 0
	page 1
	page 2
	page 3
	page 4
10,468	page 5
12,287	

frame number		valid-invalid bit
0	2	v
1	3	v
2	4	v
3	7	v
4	8	v
5	9	v
6	0	i
7	0	i

page table

0	
1	
2	page 0
3	page 1
4	page 2
5	
6	
7	page 3
8	page 4
9	page 5
	⋮
	page <i>n</i>





Shared Pages

■ Shared code

- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

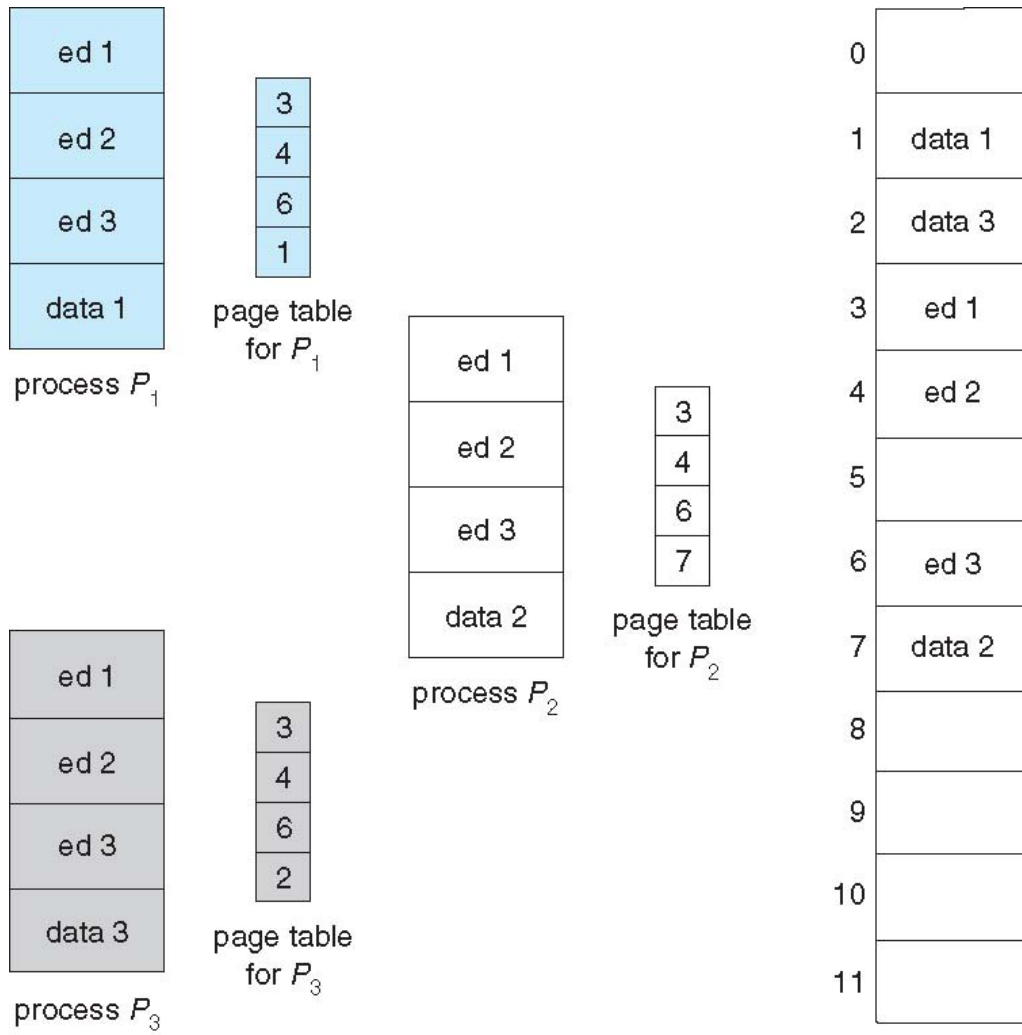
■ Private code and data

- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space





Shared Pages Example





Structure of the Page Table

- Memory structures for paging can get huge using straightforward methods
 - Consider a 32-bit logical address space as on modern computers
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes $\rightarrow$ 4 MB of physical address space / memory for page table alone
 - ▶ That amount of memory used to cost a lot
 - ▶ Don't want to allocate that contiguously in main memory
- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables





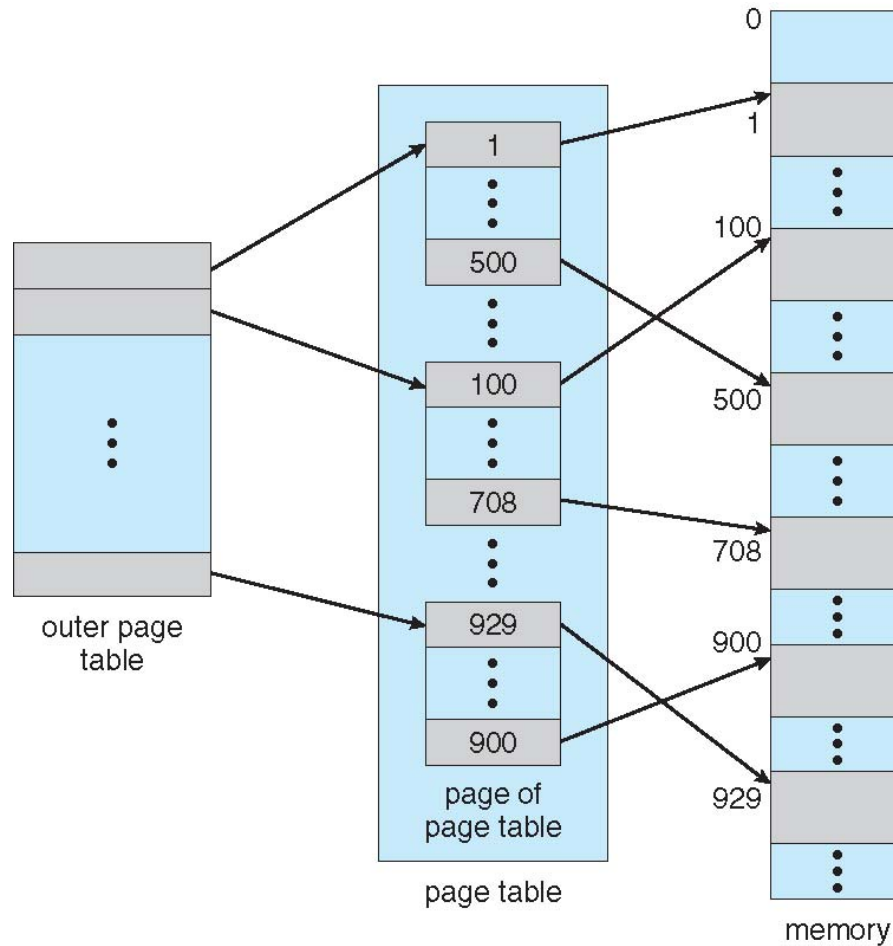
Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table





Two-Level Page-Table Scheme





Two-Level Paging Example

A memory management scheme where the page table is split into two levels to efficiently handle large logical address spaces.

- A logical address (on 32-bit machine with 1K page size) is divided into:
 - a page number consisting of 22 bits
 - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
 - a 12-bit page number
 - a 10-bit page offset

- Thus, a logical address is as follows:

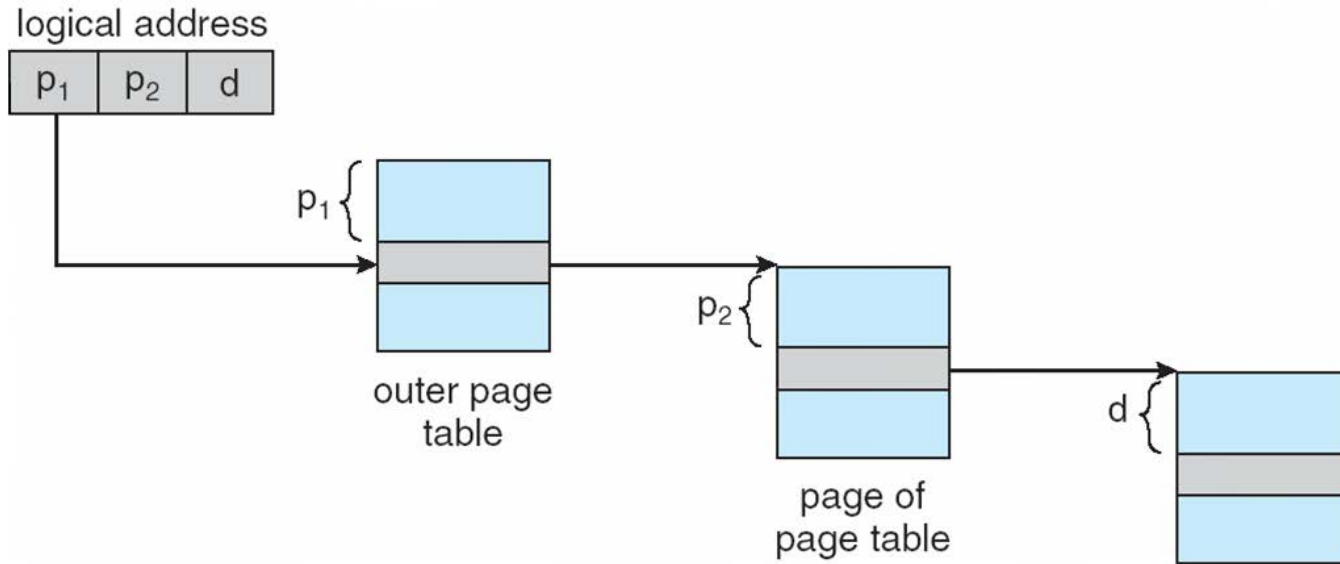
page number		page offset
p_1	p_2	d
12	10	10

- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**





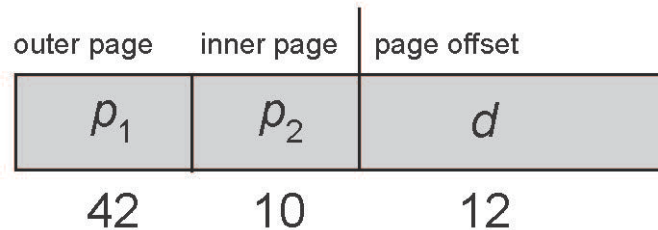
Address-Translation Scheme





64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2^{nd} outer page table
- But in the following example the 2^{nd} outer page table is still 2^{34} bytes in size
 - ▶ And possibly 4 memory access to get to one physical memory location





Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12





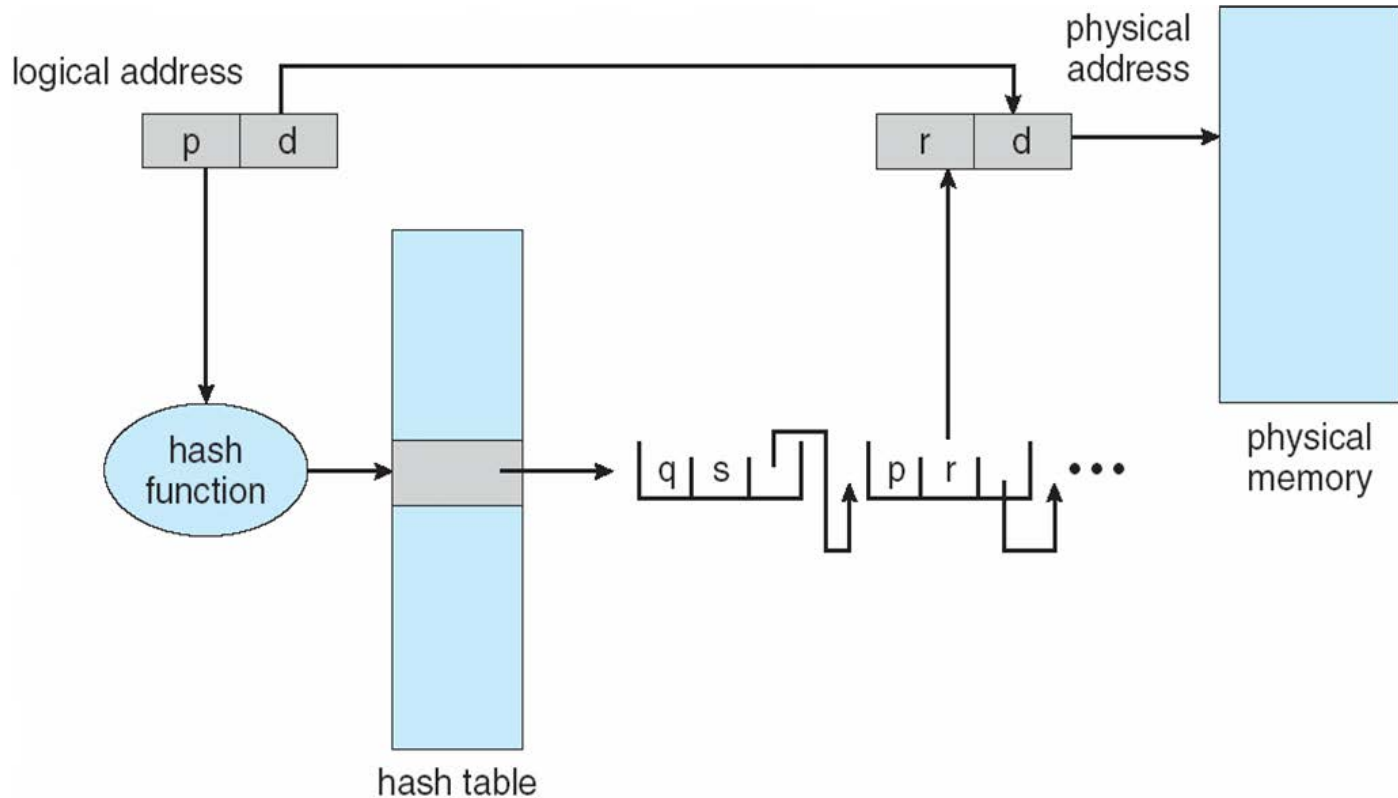
Hashed Page Tables

- Common in address spaces > 32 bits
- The virtual page number is hashed into a page table
 - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
 - If a match is found, the corresponding physical frame is extracted
- Variation for 64-bit addresses is **clustered page tables**
 - Similar to hashed but each entry refers to several pages (such as 16) rather than 1
 - Especially useful for **sparse** address spaces (where memory references are non-contiguous and scattered)





Hashed Page Table





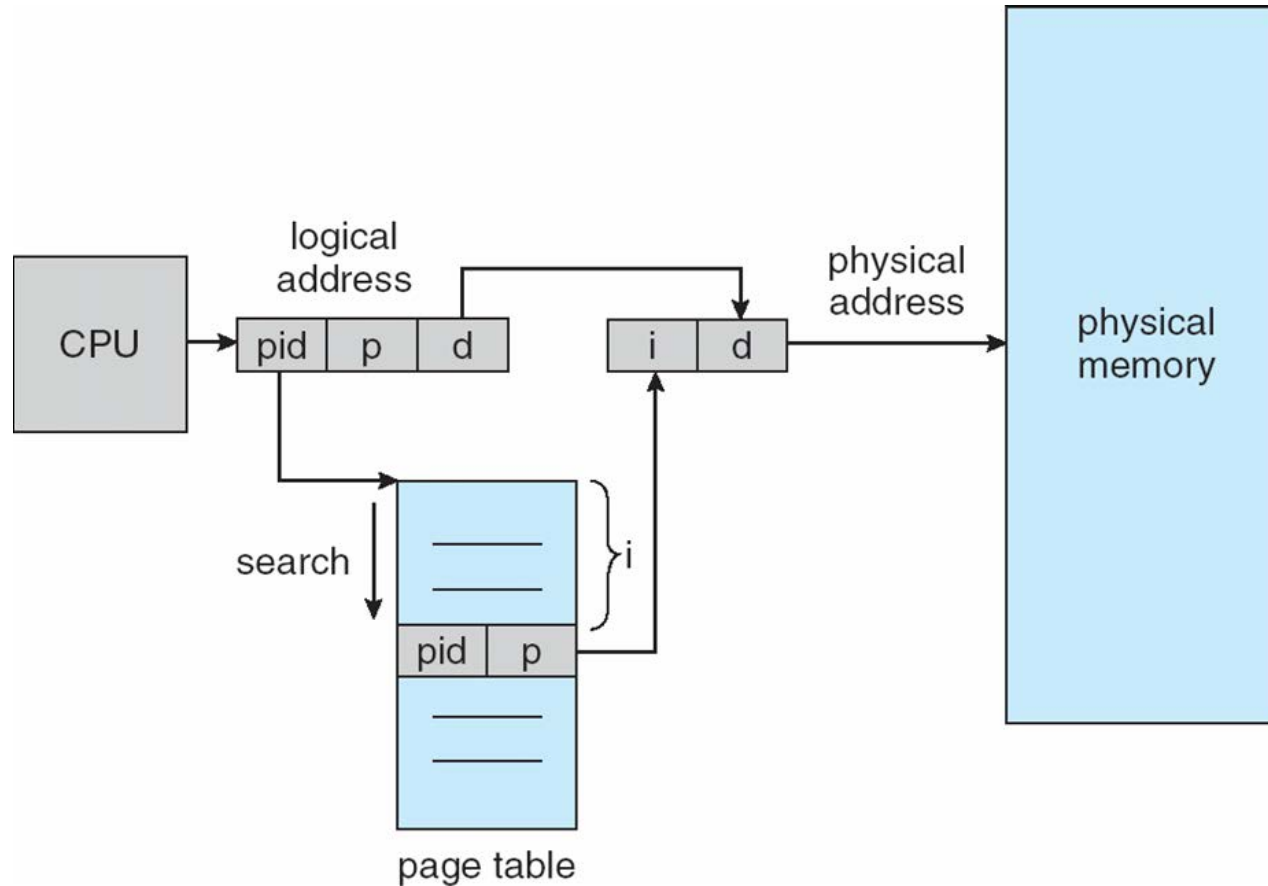
Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
 - TLB can accelerate access
- But how to implement shared memory?
 - One mapping of a virtual address to the shared physical address





Inverted Page Table Architecture





Oracle SPARC Solaris

- Consider modern, 64-bit operating system example with tightly integrated HW
 - Goals are efficiency, low overhead
- Based on hashing, but more complex
- Two hash tables
 - One kernel and one for all user processes
 - Each maps memory addresses from virtual to physical memory
 - Each entry represents a contiguous area of mapped virtual memory,
 - ▶ More efficient than having a separate hash-table entry for each page
 - Each entry has base address and span (indicating the number of pages the entry represents)





Oracle SPARC Solaris (Cont.)

- TLB holds translation table entries (TTEs) for fast hardware lookups
 - A cache of TTEs reside in a translation storage buffer (TSB)
 - ▶ Includes an entry per recently accessed page
- Virtual address reference causes TLB search
 - If miss, hardware walks the in-memory TSB looking for the TTE corresponding to the address
 - ▶ If match found, the CPU copies the TSB entry into the TLB and translation completes
 - ▶ If no match found, kernel interrupted to search the hash table
 - The kernel then creates a TTE from the appropriate hash table and stores it in the TSB, Interrupt handler returns control to the MMU, which completes the address translation.





Example: The Intel 32 and 64-bit Architectures

- Dominant industry chips
- Pentium CPUs are 32-bit and called IA-32 architecture
- Current Intel CPUs are 64-bit and called IA-64 architecture
- Many variations in the chips, cover the main ideas here





Example: The Intel IA-32 Architecture

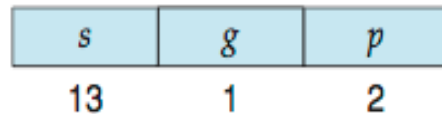
- Supports both segmentation and segmentation with paging
 - Each segment can be 4 GB
 - Up to 16 K segments per process
 - Divided into two partitions
 - ▶ First partition of up to 8 K segments are private to process (kept in **local descriptor table (LDT)**)
 - ▶ Second partition of up to 8K segments shared among all processes (kept in **global descriptor table (GDT)**)





Example: The Intel IA-32 Architecture (Cont.)

- CPU generates logical address
 - Selector given to segmentation unit
 - ▶ Which produces linear addresses

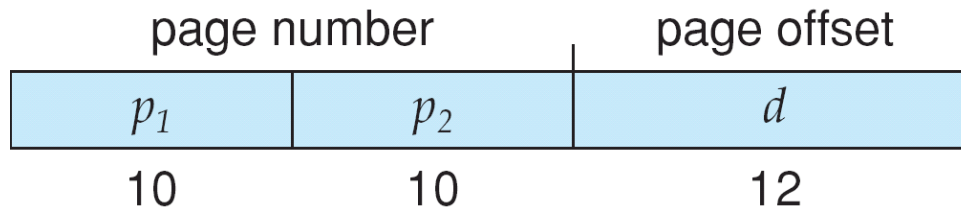
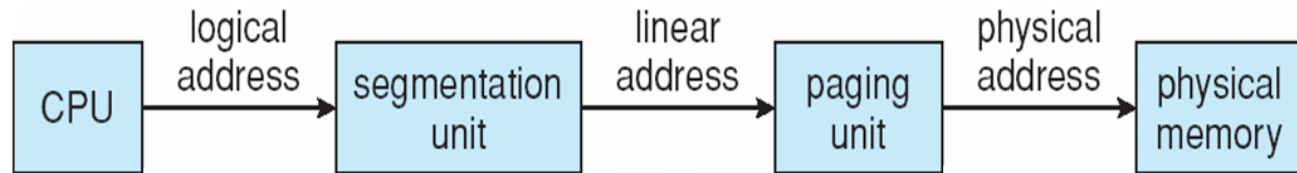


- Linear address given to paging unit
 - ▶ Which generates physical address in main memory
 - ▶ Paging units form equivalent of MMU
 - ▶ Pages sizes can be 4 KB or 4 MB



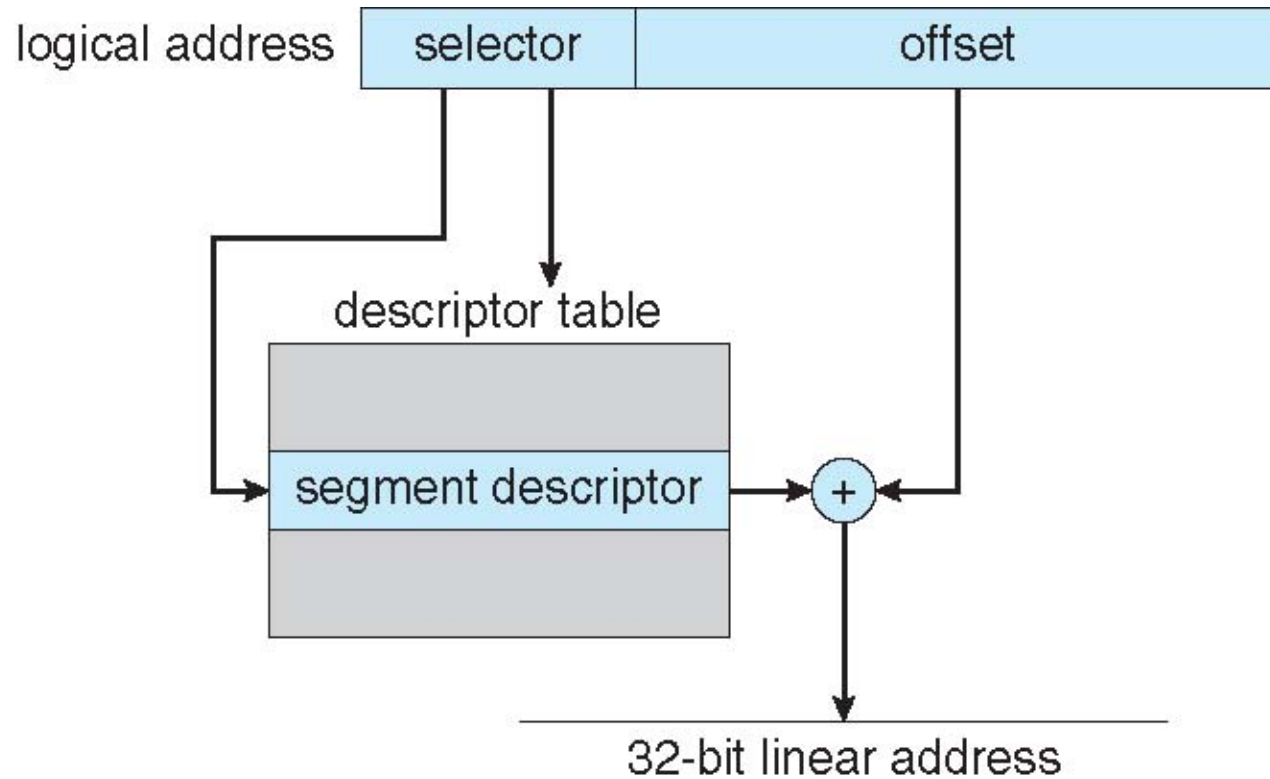


Logical to Physical Address Translation in IA-32



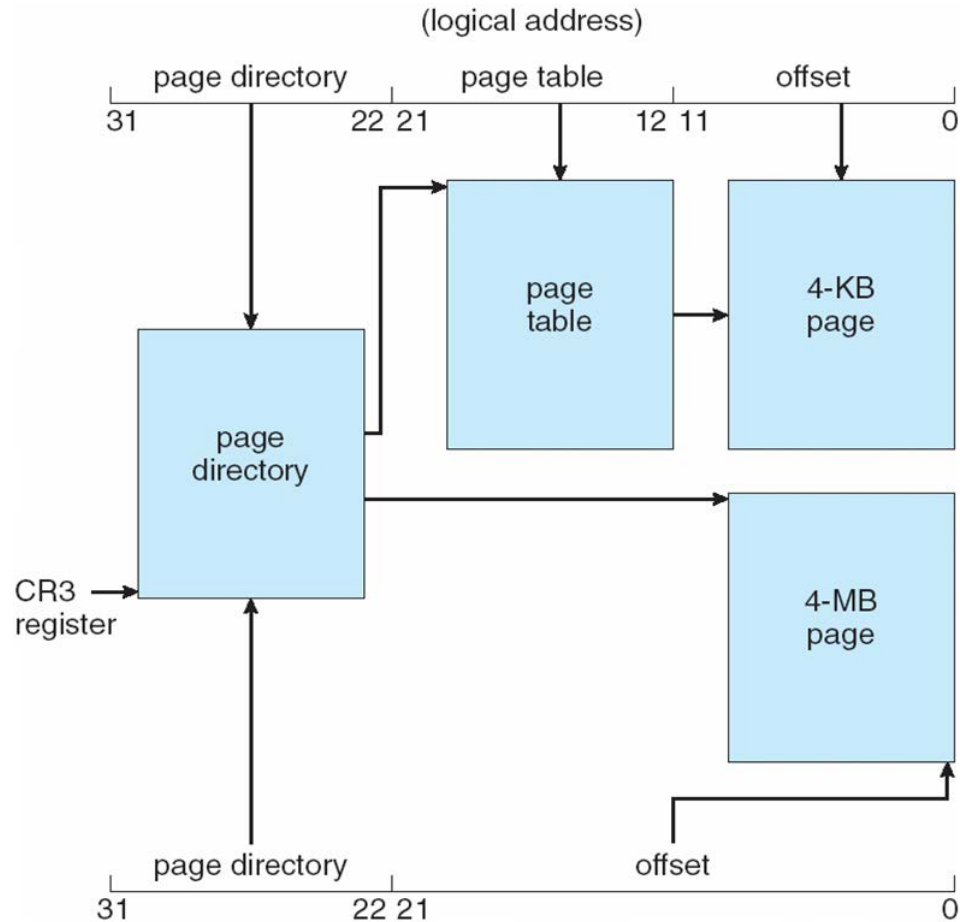


Intel IA-32 Segmentation





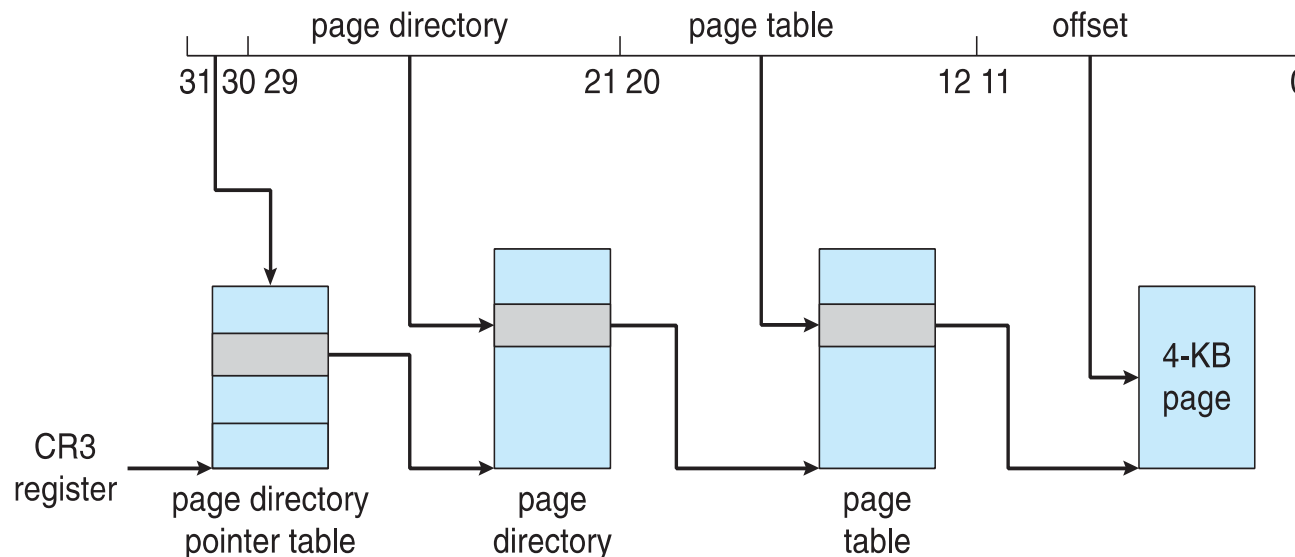
Intel IA-32 Paging Architecture





Intel IA-32 Page Address Extensions

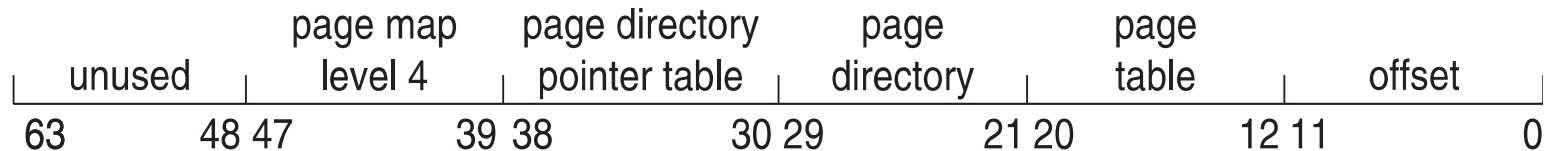
- 32-bit address limits led Intel to create **page address extension (PAE)**, allowing 32-bit apps access to more than 4GB of memory space
 - Paging went to a 3-level scheme
 - Top two bits refer to a **page directory pointer table**
 - Page-directory and page-table entries moved to 64-bits in size
 - Net effect is increasing address space to 36 bits – 64GB of physical memory





Intel x86-64

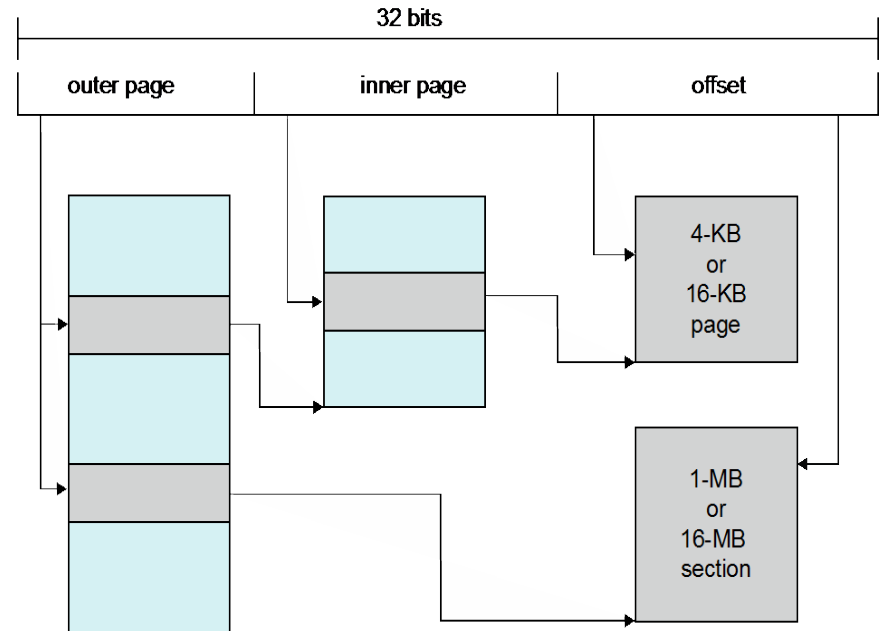
- Current generation Intel x86 architecture
- 64 bits is ginormous (> 16 exabytes)
- In practice only implement 48 bit addressing
 - Page sizes of 4 KB, 2 MB, 1 GB
 - Four levels of paging hierarchy
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits





Example: ARM Architecture

- Dominant mobile platform chip (Apple iOS and Google Android devices for example)
- Modern, energy efficient, 32-bit CPU
- 4 KB and 16 KB pages
- 1 MB and 16 MB pages (termed **sections**)
- One-level paging for sections, two-level for smaller pages
- Two levels of TLBs
 - Outer level has two micro TLBs (one data, one instruction)
 - Inner is single main TLB
 - First inner is checked, on miss outers are checked, and on miss page table walk performed by CPU



End of Chapter 8

